Promo Finder: picking promo cuts straight from the scripts

Kuku AI Generalist Assignment, Problem 1 (Find the Promos)

Live app: promo.expediva.in | Code: github.com/Vivek24411/promo-finder | Raw outputs and video: in the form

Why I picked Problem 1

Right now an editor watches every episode of every show to pick promo moments, and Kuku releases a new show every week. That is a cost that repeats forever, the input pack was ready to use, and the output can actually be checked: a cut either lands inside a spoken line or it doesn't, and "a hook in the first 3 seconds" can be tested clip by clip. Problem 2 is outside Kuku's business, and for Problem 3 I couldn't collect real user evidence in 48 hours.

What I assumed

- The tool reads the screenplay only. It can't see acting, music or pacing, so an editor still makes the final call.
- A promo is 20-60 s in total: one stretch, or two moments joined in story order. A cut never lands in the middle of a spoken line.
- "Hook in the first 3 s" means a beat that starts there and mostly plays there. A line from 1.0 s to 3.8 s counts; one that starts at 2.9 s and runs to 6 s doesn't.
- 25 is the most promos I return, not a target. If only 9 clips pass, the list has 9.

How it works

1. **Parse (code).** Every script becomes beats with fixed IDs and exact times. All 3,927 timestamp lines in the pack were read.
2. **First pass (free model).** Gemini 3.5 Flash-Lite reads every episode in overlapping chunks and tags hooks, reveals, threats and open questions by beat ID, each with an exact quote from the script.
3. **Candidates (code).** Code builds every legal window (68,447 for this show), scores them and sends the 60 most promising, plus a few random audit picks, to the final review.
4. **Final review (paid model).** Claude Haiku 5.5 sees one clip at a time, as someone who has never seen the show, and answers three things: does something grab you in the first 3 s, does the ending leave a real question open, and can you follow it without context? Every answer has to quote the script.
5. **Checks (code).** The quotes must match the script exactly, the hook beat must land in the first 3 s, the ending evidence must come from the last 5 s and the clip must not answer its own question. Each clip ends up Recommended, Needs review or Rejected, with the reason shown, and the editor approves or rejects it in the app.

The model never writes a timestamp. It only points at beat IDs, and code looks the times up, so a cut can't drift.

Why one free model and one paid model

The two stages have very different shapes. The first pass reads all 50 episodes (about 56 calls and 200k input tokens per show), but it only has to notice candidate moments; the strict judging happens later. That is most of the tokens, and a free tier handles it fine. The final review is only about 72 calls per show, but its answer decides what gets recommended, so that is where I spend money. I used the smallest current Claude model; Sonnet costs about 20 times more per token, so I'd only move up if editors disagree with Haiku's calls.

The paid part really is small: one full 50-episode run cost about $0.10 of Claude usage (under Rs 10), and the whole day of debugging, five full runs, came to under $0.50. Both stages can be swapped from the app (provider, model and your own API key, which stays only in the browser session), so a stronger model can be plugged in during the interview without touching code. A budget guard ($2 per run by default) stops calls before the limit, and refuses to call a model whose price isn't set.

Scaling to every show, and cutting cost further

Today one show takes about 11 minutes and $0.10. Most of that time is the free tier's rate limit (12 requests a minute), not compute; on a paid tier with parallel calls it would be a couple of minutes. To cut cost at volume:

- Cache everything (already built): a rerun, or a show with one new episode, only pays for what changed.
- Send the final review through the Batch API at about half price, since editors don't need it in real time.
- Cache the instruction block that is identical in every review prompt (prompt caching), a big share of the input.
- Send fewer clips to the paid stage. Editor approvals will show how many of the 60 are actually needed.
- At very high volume, run an open model for the first pass on our own GPU.

How I know it's good, and where it isn't

- **Tests:** 140 automated tests with a fake model, so they cost nothing: mid-line cuts, late hooks, endings that answer their own question, invented quotes, confusing cuts, budget limits, retries and model swaps (including a check that typed keys never reach any output file).
- **Full run on the pack:** all 50 episodes processed and 72 clips reviewed. 45 passed every check, 25 are returned (from 25 different episodes), 19 went to Needs review and 8 were rejected, each with a reason.
- **Against my own picks:** I marked 20 strong moments before looking at results. The top 25 cover 9 of them; my earlier version covered 12. That gap is within noise, and I'd rather show fewer clips that each pass their checks than more that fail them (the earlier version shipped 7 of 25 that failed its own checks).
- **Where it fails:** only 72 of 1,216 candidate windows get the paid review, so good moments can be missed; it can't judge acting or music; the judge is still generous about hooks; I only had one show to test on; and no editor has rated the output yet. Those ratings are the next thing I'd collect.

Where the editor stays in the loop

The tool does the first pass, not the final decision. Every clip opens with its timestamps, the script excerpt, the evidence for the hook and the ending, why it ranks where it does, and one concern. The editor approves or rejects it, and those decisions are saved separately from the model's answer, so over time they become labels I can use to tune how many clips go to review and which model is worth paying for.

Running it on a new show

Open the app, upload the episode files, a folder or a .zip, and check the parse summary (episodes, beats, and the share of timestamp lines it could read; below 99% it stops instead of quietly skipping beats). Then press Find promos. To try a different model, open "Models and API keys", pick the provider for either stage, paste the key and its prices, and run again. If only the final review changes, the first-pass answers come from cache.

AI tools I used, and one place they got it wrong

I built this with Claude Code and Codex, and the pipeline itself uses Gemini and Claude. The clearest mistake: Codex wrote a checker that demanded one exact quote for every beat the judge cited, but the example reply it put in the prompt had no quotes field. The model copied the example, so my first real run recommended only 3 clips. I didn't trust that number, replayed the saved replies, and found that all 1,110 quotes Claude had given were exact. The bug was in the contract, not the model. After fixing it, and seven smaller issues I found the same way, 45 clips passed, with the check against invented quotes just as strict as before.